

Logic

Logic topics

- Foundations
- Coding
- Tools for algorithms
- Flow charts

warm-up

- Code understanding
- Requirements definition
- Simple flows and readability

Coding - arrays

- Counting
- Research (linear)
- Minimum, Maximum, Average
- Conditional count
- Second max & others...

Coding - strings

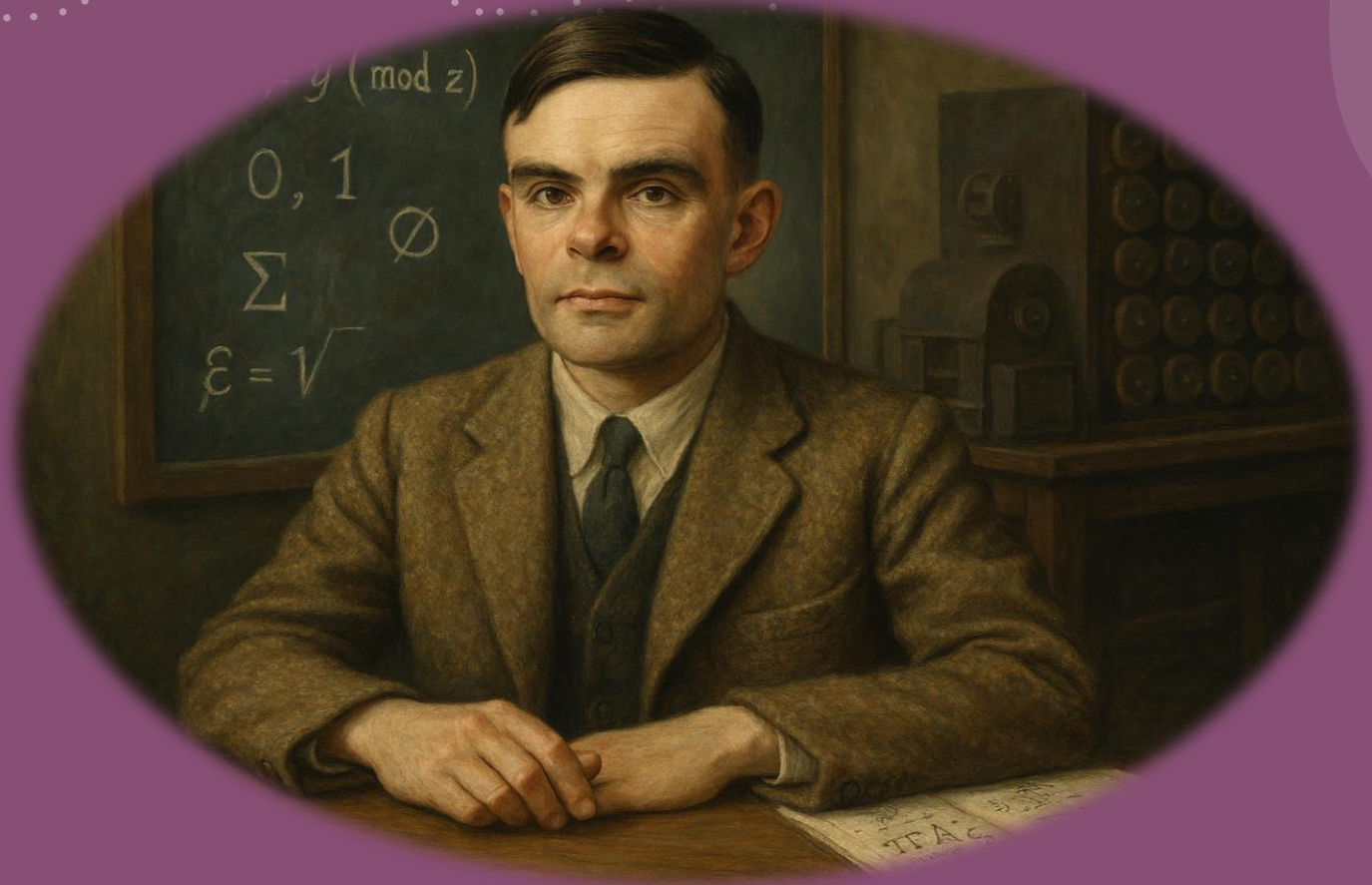
- Words starting
- Replace letters
- Index of a letter, of a word & others...

Coding - complex problems

- Array exploration
- Palindrome
- Fibonacci & others...
- Replace

Alan Turing

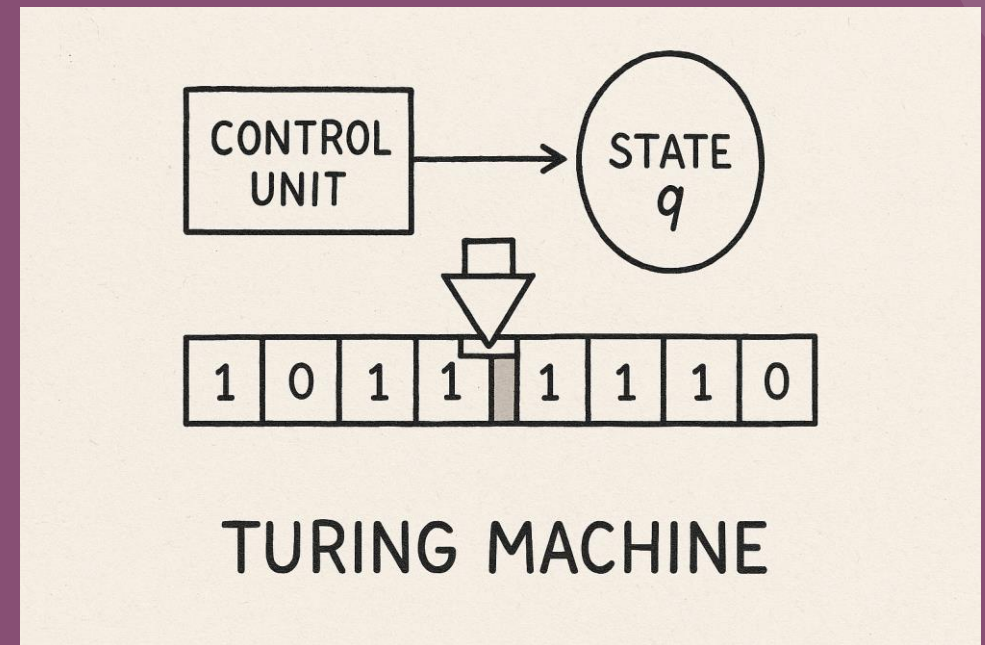
The father of algorithm-based machines
and theoretical computer science



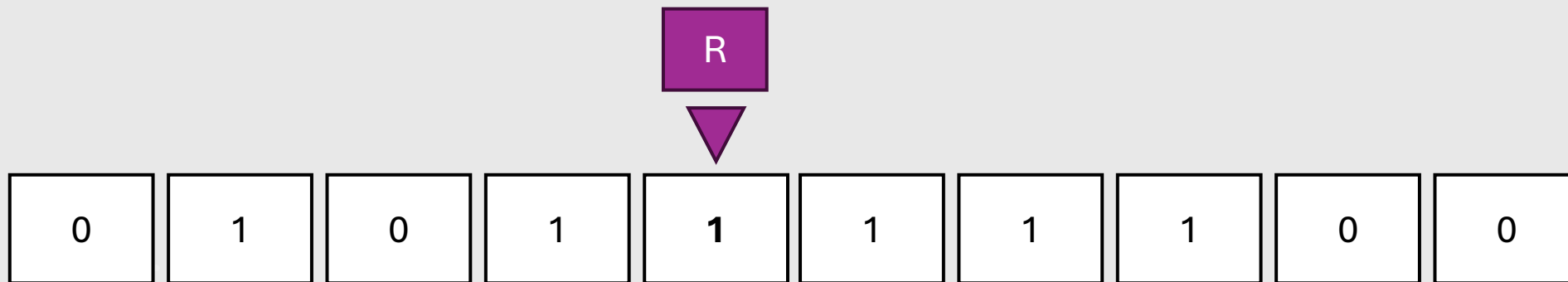
Turing's Machine

The Turing machine is a theoretical model of computation.

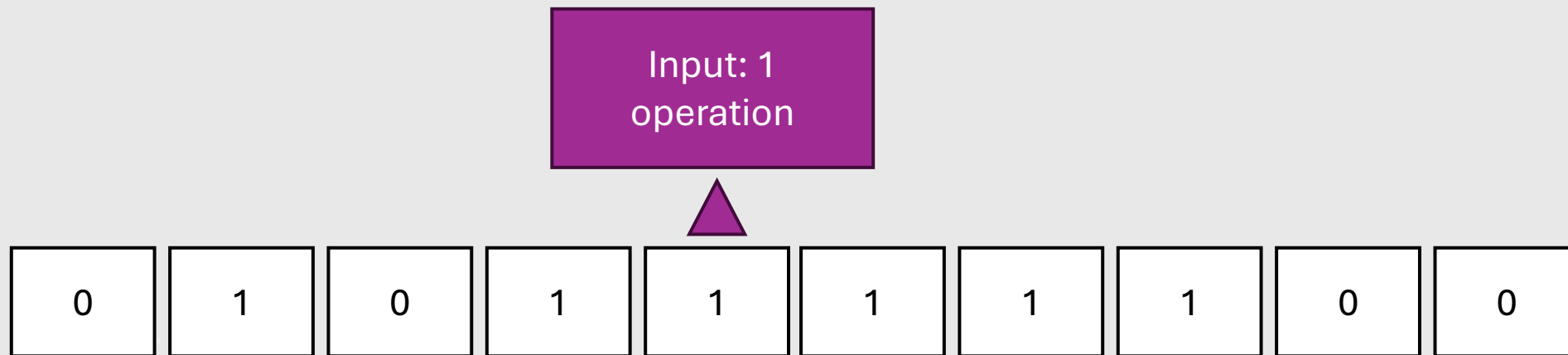
It is an abstract example of basic computer architecture and served as a model for the design of modern computation units (CPUs).



Turing's Machine



Turing's Machine

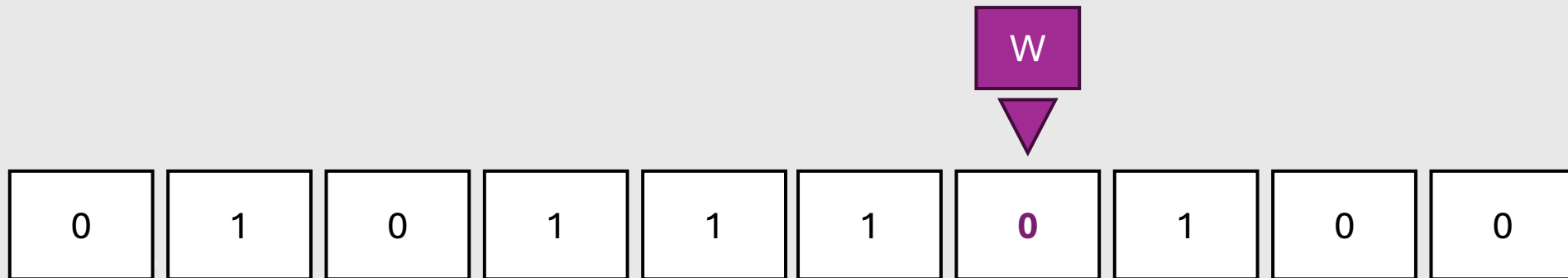


Turing's Machine

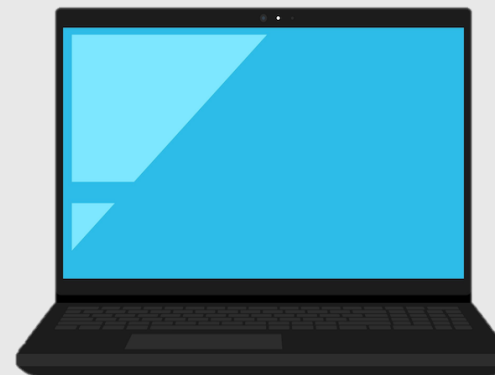
Ouput: 0
operation

0	1	0	1	1	1	1	1	0	0
---	---	---	---	---	---	---	---	---	---

Turing's Machine



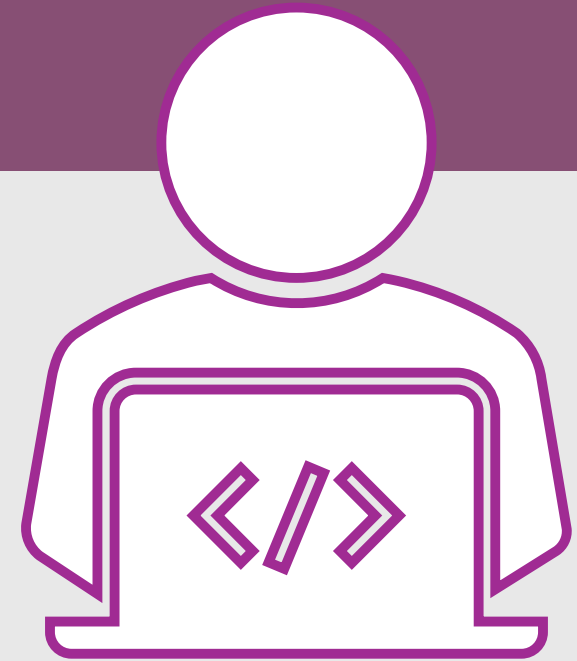
1940s – Turing's contribute



Coding

Electric machines designed in Turing's era were designed with a specific purpose – like a calculator.

His revolutionary idea was to think about an «electric machine» that could be used for any kind of operation, by **programming an algorithm**.



Operations

Assignment



Unary operator, then assignment



Binary operator, then assignment



Operations

Assignment



Unary operator, then assignment

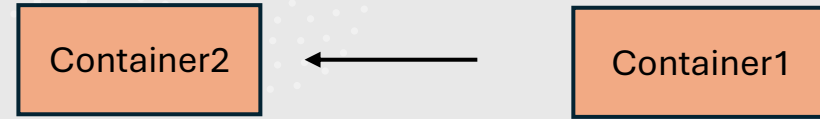


Binary operator, then assignment

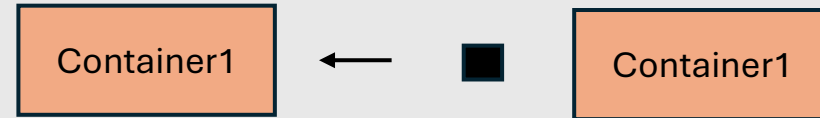


Operations

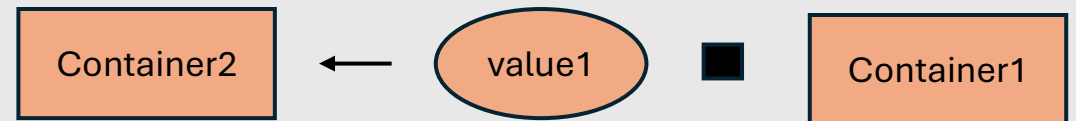
Assignment



Unary operator, then assignment



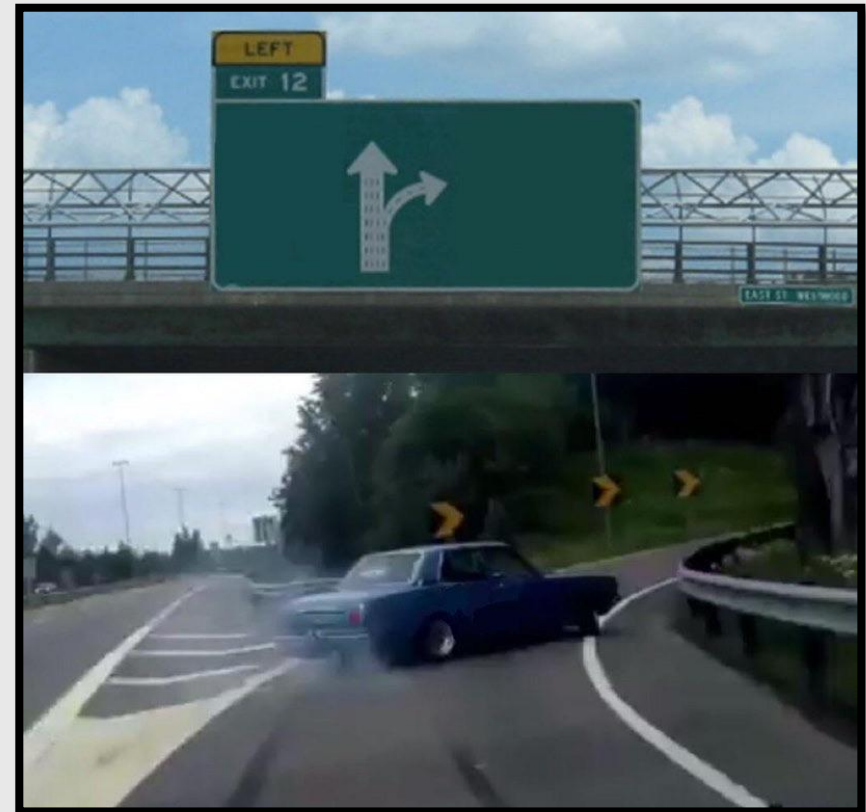
Binary operator, then assignment



Filter



Branches the execution based on condition



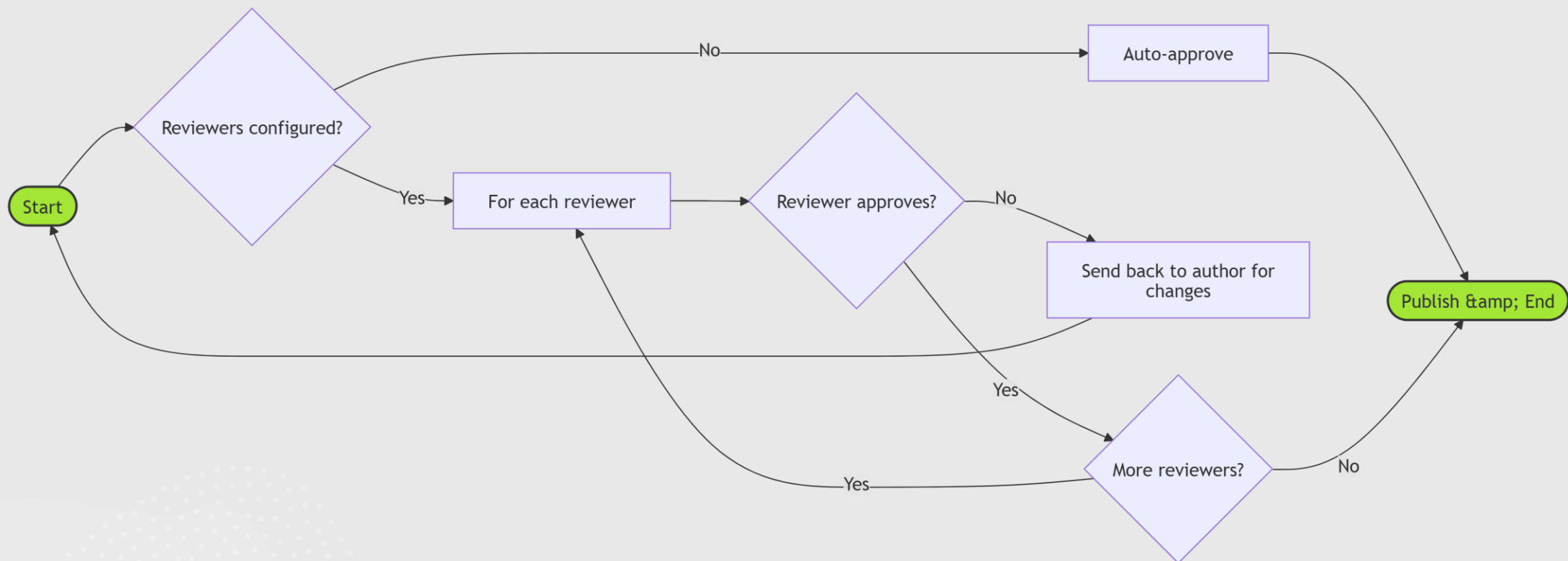
Loop



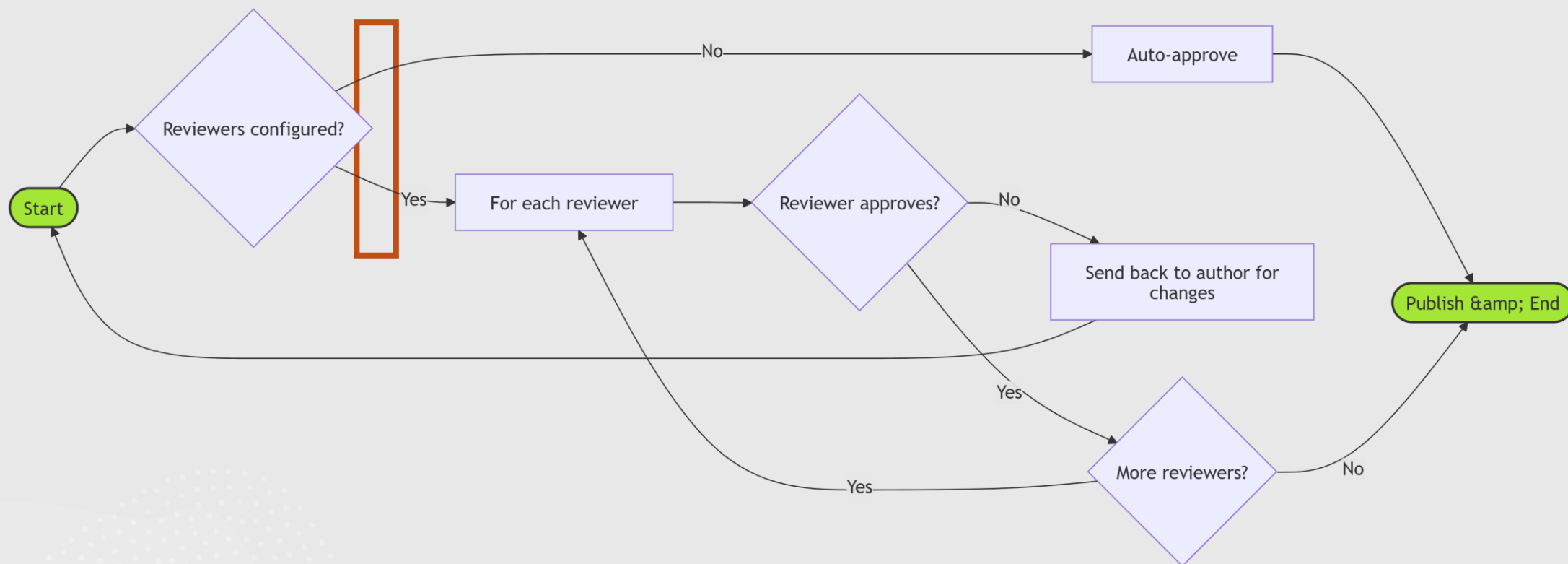
Replicates the execution based on a condition



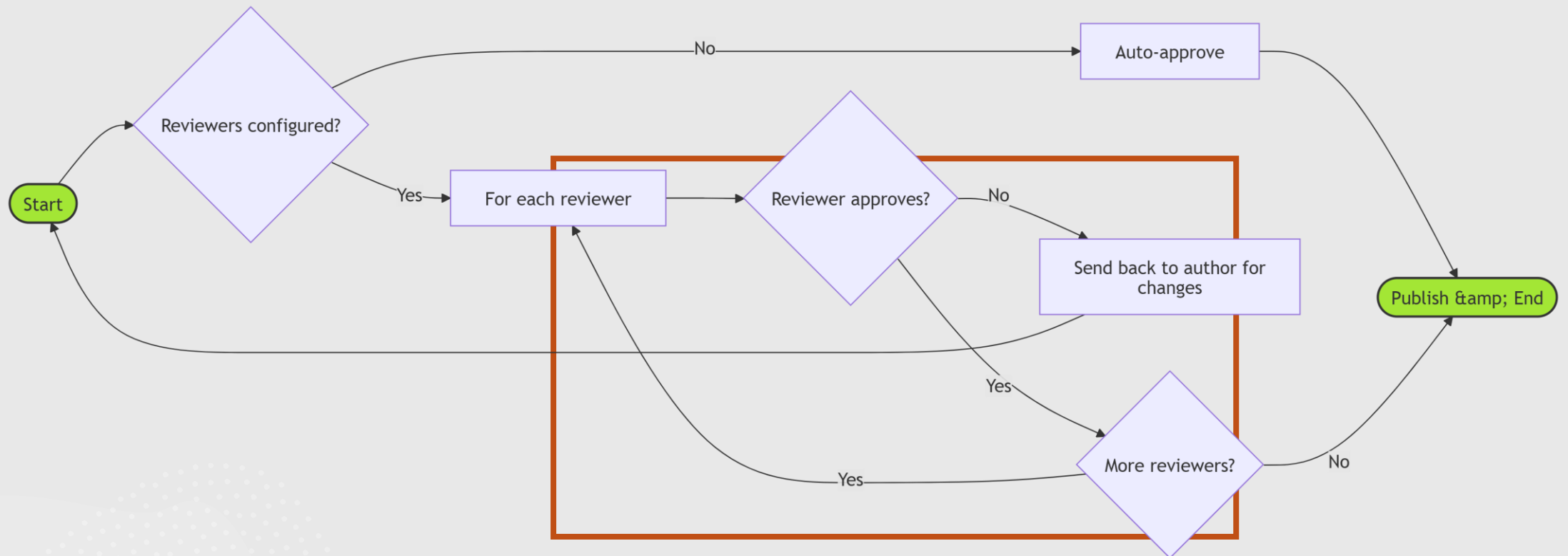
Approval flow-chart



Branches

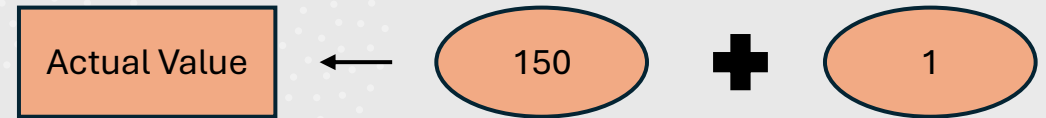


Loop



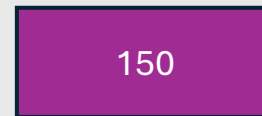
Testing logic

Expression to test



actual value: 151, produced executing the function

Expected value



TEST FAILED



TEST PASSED

If expected value we set is the same of the actual value, the test is passed.

Else it is failed

Logic topics

- Foundations
- Coding
- Tools for algorithms
- Flow charts



warm-up

- Code understanding
- Requirements definition
- Simple flows and readability

Coding - arrays

- Counting
- Research (linear)
- Minimum, Maximum, Average
- Conditional count
- Second max & others...

Coding - strings

- Words starting
- Replace letters
- Index of a letter, of a word & others...

Coding - complex problems

- Array exploration
- Palindrome
- Fibonacci & others...
- Replace

Chapter

Chapter 0

Coding

Level

EASY

MEDIUM

HARD

OMG

No
massive LLMs use



Compute your future salary!

Chapter 0

Some hackers have leaked the code used to compute MSC's employees salaries.

Can you please help in explaining how the salary is calculated?



Compute your future salary!

SalarioTestInputOutput contains an array of «OreLavorate» and the expected «Salary».

Can you please help identifying the expected salary given the code?

Now expected values are setup at zero but all the tests are indeed failing 😞

Feel free to remove 0 and insert the with correct salary values 😊

```
var listaDiTest = new SalarioTestInputOutput[]  
{  
    new SalarioTestInputOutput(new int[] { 1, 0, 0, 0}, 0),  
    new SalarioTestInputOutput(new int[] { 1, 1, 1, 1}, 0),  
    new SalarioTestInputOutput(new int[] { 1, 8, 8, 0, 0}, 0),  
    new SalarioTestInputOutput(new int[] { 0, 10, 10, 0}, 0),  
    new SalarioTestInputOutput(new int[] { 1, 0, 1, 0}, 0),  
    new SalarioTestInputOutput(new int[] { 1, 1, 1, 1, 1, 1, 1}, 0),  
};
```

Compute your future salary!

Now let's try to formalize the rules.

Check the code and try to list some bullet-points formalizing the *requirements*

Requirements should be true for any kind of input! (input is *oreLavorate*)

Requirements

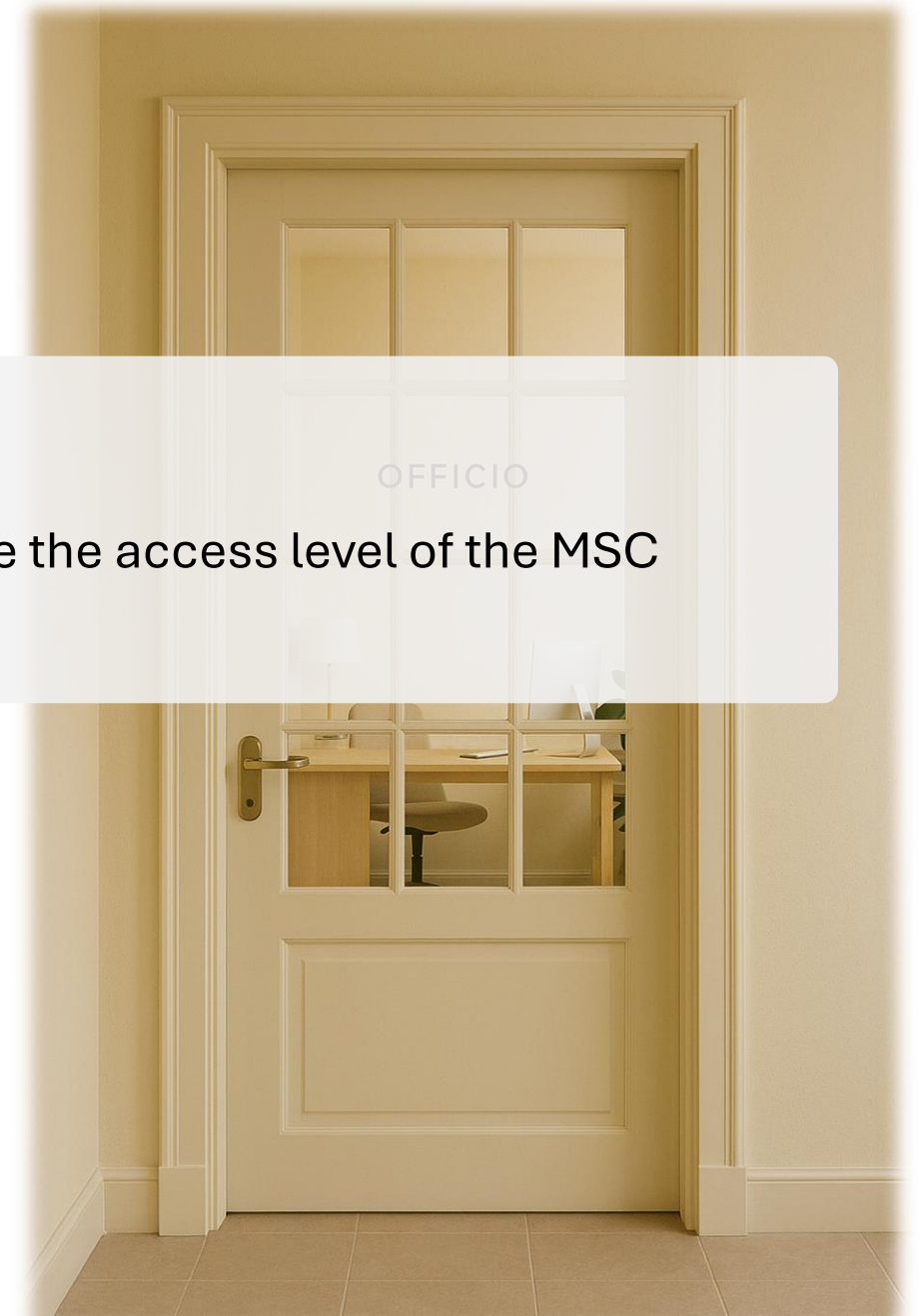
- Each hour worked at the office is paid according to the base hourly rate provided as input.
- For every day of absence, a deduction equivalent to 2 hours of pay is applied to the salary.
- Overtime is paid at 150% of the regular hourly rate.
- A fidelity bonus is added based on the number of days worked. Overtime days grant double the usual daily fidelity bonus.
- Working on Sundays automatically grants a fixed bonus (1000).
- After calculating the above, the total fidelity bonus acts as a multiplier for the computed salary.
- If more than 5 consecutive days are worked, a continuity bonus is added, calculated according to the number of consecutive days worked.
- At the end, the compute salary is rounded-up at the second decimal digit.

Office gates

Chapter 1

We want to secure the access to MSC Technology office.

Given a series of parameters, we will build a logic to define the access level of the MSC employees.



Input

- Classic checks
- Biometric checks
- Profile checks
- Disaster States

```
12 references
public record class ControlloAccesso
{
    // Controlli classici
    12 references
    public bool BadgeValido { get; set; }
    11 references
    public string CodiceInserito { get; set; } = string.Empty;
    11 references
    public int LivelloAutorizzazione { get; set; }

    // Controlli biometrici
    11 references
    public bool RetinaValidata { get; set; }
    11 references
    public bool CodiceDnaCorretto { get; set; }
    11 references
    public bool ImprontaDigitaleCorretta { get; set; }

    // Controlli sul profilo
    11 references
    public bool OrarioConsentito { get; set; }
    11 references
    public double GiorniInAzienda { get; set; }
    12 references
    public bool LavoraATorino { get; set; }

    // Stati di emergenza
    11 references
    public bool StatoDiBlackout { get; set; }
    11 references
    public bool StatoDiEmergenza { get; set; }
}
```


Classic checks



Badge



Authorization level



Code

Biometric checks



Retina



DNA code



Fingerprint



To simplify, biometrics are validated or not validated.

Profile checks



Days in the company



Allowed time of entrance



Works in Turin

Disaster states



Emergency



Blackout



To simplify, disasters are occurring or not occurring.

Access ways

can be accessed if

Main access

badge, authorization level ≥ 3 and
validated-biometric count ≥ 2

Secondary access

badge, code = «1970» and validated-
biometric count ≥ 1

Emergency access

emergency and non-blackout

Output

- Access level:
 - «access allowed», *code 1*
 - «access under monitoring», *code 2*
 - «access forbidden», *code 3*

```
15 references  
public enum Accesso {  
    Consentito,  
    Sorvegliato,  
    Negato  
}
```

Access levels

«access allowed», code 1

can be applied to

all the access ways

and needs

- Allowed clock-in time
- **Does not work** in Turin
- Days in the company are more than 5

Access levels

«accesso sorvegliato», code 2

can be applied to

emergency access

and needs

Works in Turin

Access levels

«Access forbidden», code 3

Otherwise, access is denied.

Goals



Correct code
passing all
the tests

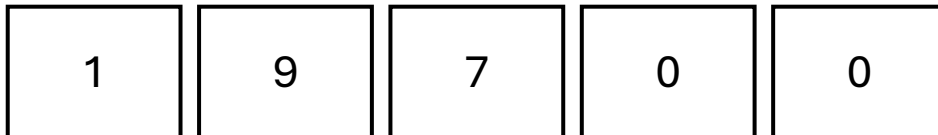


Correct and
readable code
passing all the
tests

Arrays

Chapter 2

- Arrays are an ordered set of items of the same type
- Iterating over a collection means inspecting items one by one
- Declaring an array means declaring a contiguous number of data of the same type





Conta

Level

EASY

```
public static int Conta(int[] collezione)
```

Counts the number of elements.

Cerca

Level

EASY

```
public static int Cerca(int[] collezione, int obiettivo)
```

Looks for the target elements in the collection. Returns true if it is present, else false.

Cerca Minimo

Level

EASY

```
public static int CercaMinimo(int[] collezione)
```

Returns the minimum value of the collection.

Cerca Massimo

Level

EASY

```
public static int CercaMassimo(int[] collezione)
```

Returns the maximum value of the collection.

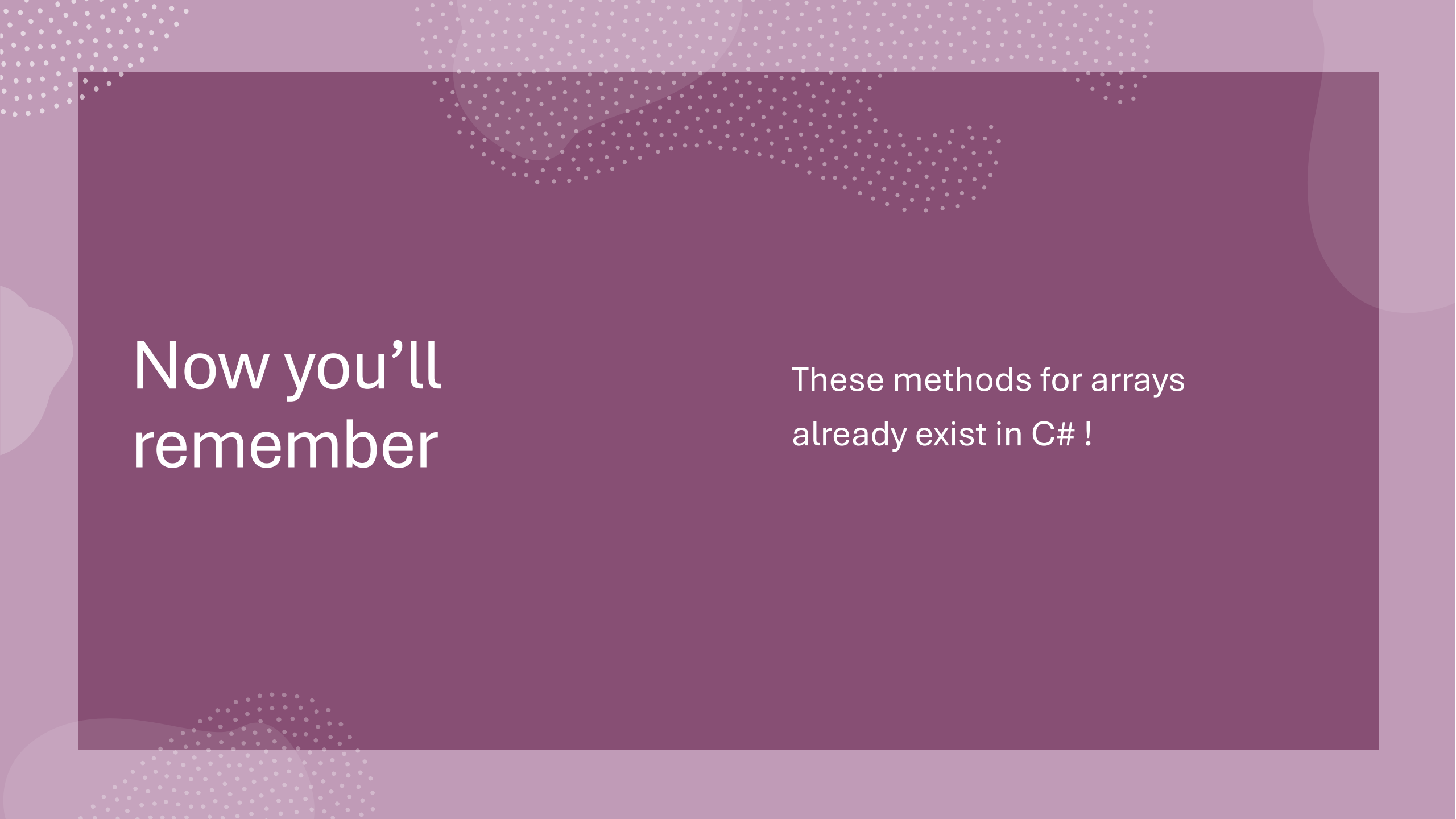
Somma

Level

EASY

```
public static int Somma(int[] collezione)
```

Returns the sum of the values of the collection.



Now you'll remember

These methods for arrays
already exist in C# !

Conta Intervallo

Level

EASY

```
public static double ContaIntervallo(List<int> collezione, int minimo, int massimo)
```

Returns the count of values in an interval.

Conta Se con opzioni

Level

MEDIUM

```
public static double ContaSe(List<int> collezione, int option)
```

Returns the count of values in an interval.

If option is 1, it counts values higher than 0.

If option is 0, it counts values equal to zero.

If option is -1, it counts values lower than zero.

Implementation: internally uses «ContaIntervallo»

Throws an exception if option is not valid.

Calcola Media

Level

MEDIUM

```
public static double CalcolaMedia(List<int> collezione)
```

Returns the average of the values of the collection.

Implementation: internally uses «Somma», «Conta»

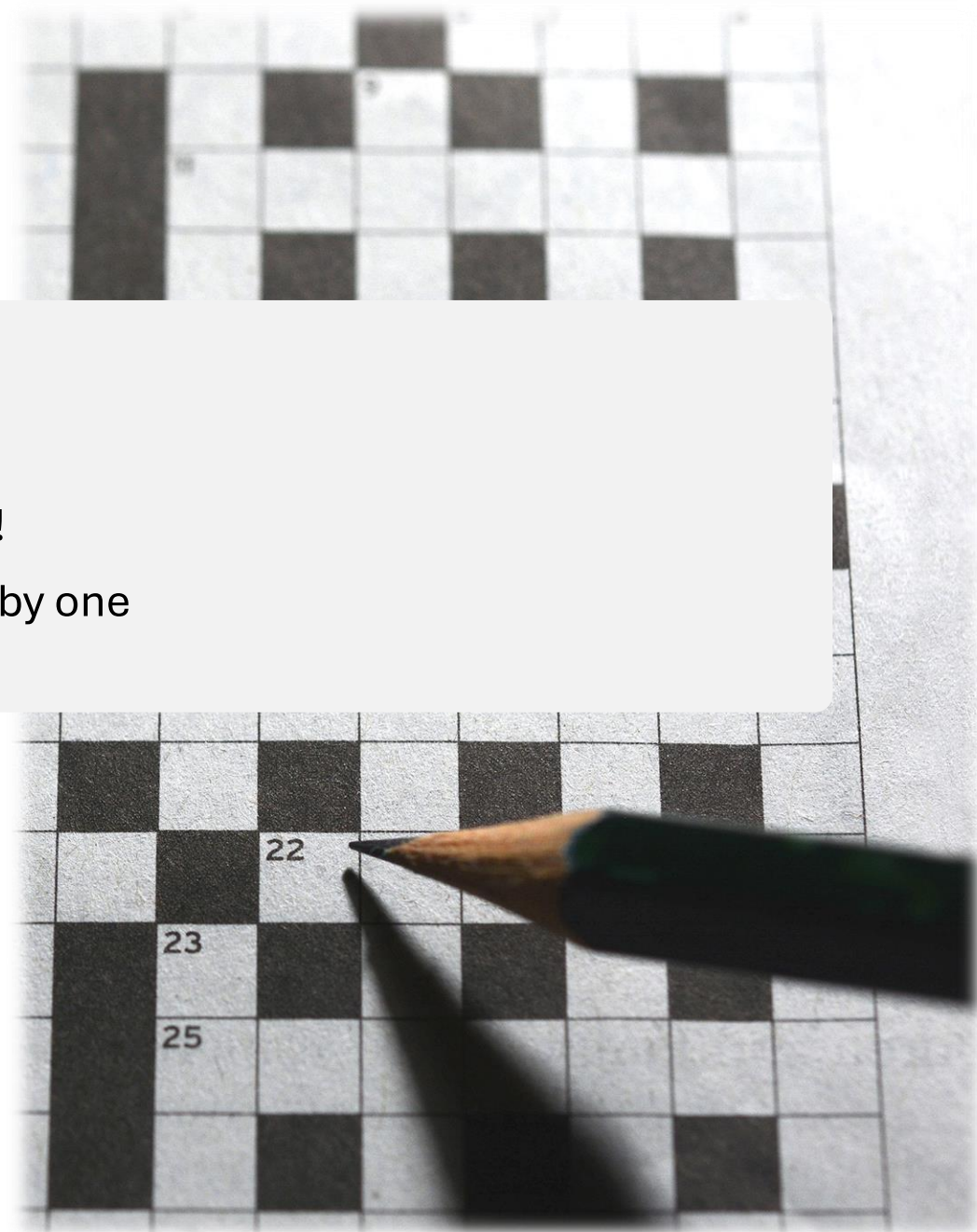
Strings

Chapter 3

- String is the type to handle text data
- Strings and characters can be compared with '=='
- Remind: Upper case and lower case letters are different!
- Iterating over a string, means inspecting characters one by one

CIAO!

C	I	A	O	!
---	---	---	---	---



Lunghezza

Level

EASY

```
public static int Lunghezza(string sorgente)
```

It returns the length of the string.

Trova Posizione Di

Level

EASY

```
public static int TrovaPosizioneDi(string sorgente, string obiettivo)
```

It returns the position of the first occurrence of a character in the source string, starting from zero.

It returns -1 if the character is not available.

Inizia Con

Level

EASY

```
public static bool IniziaCon(string sorgente, string blocco)
```

It returns true if the source string starts with the given substring.
Else it returns false.

Contiene

Level

MEDIUM

```
public static bool Contiene(string sorgente, string blocco)
```

It looks for a substring within a source string.

If the substring is contained, it returns true, else it returns false.

Trova Inizio di un blocco

Level

MEDIUM

```
public static bool TrovaInizioDi(string sorgente, string blocco)
```

It looks for a substring within a source string.

If the substring is contained, it returns the starting position of it (starting from 0)

Else it returns -1.

Rimpiazza Semplice

Level

MEDIUM

```
public static bool RimpiazzaSemplice(string sorgente, string blocco, string rimpiazzo)
```

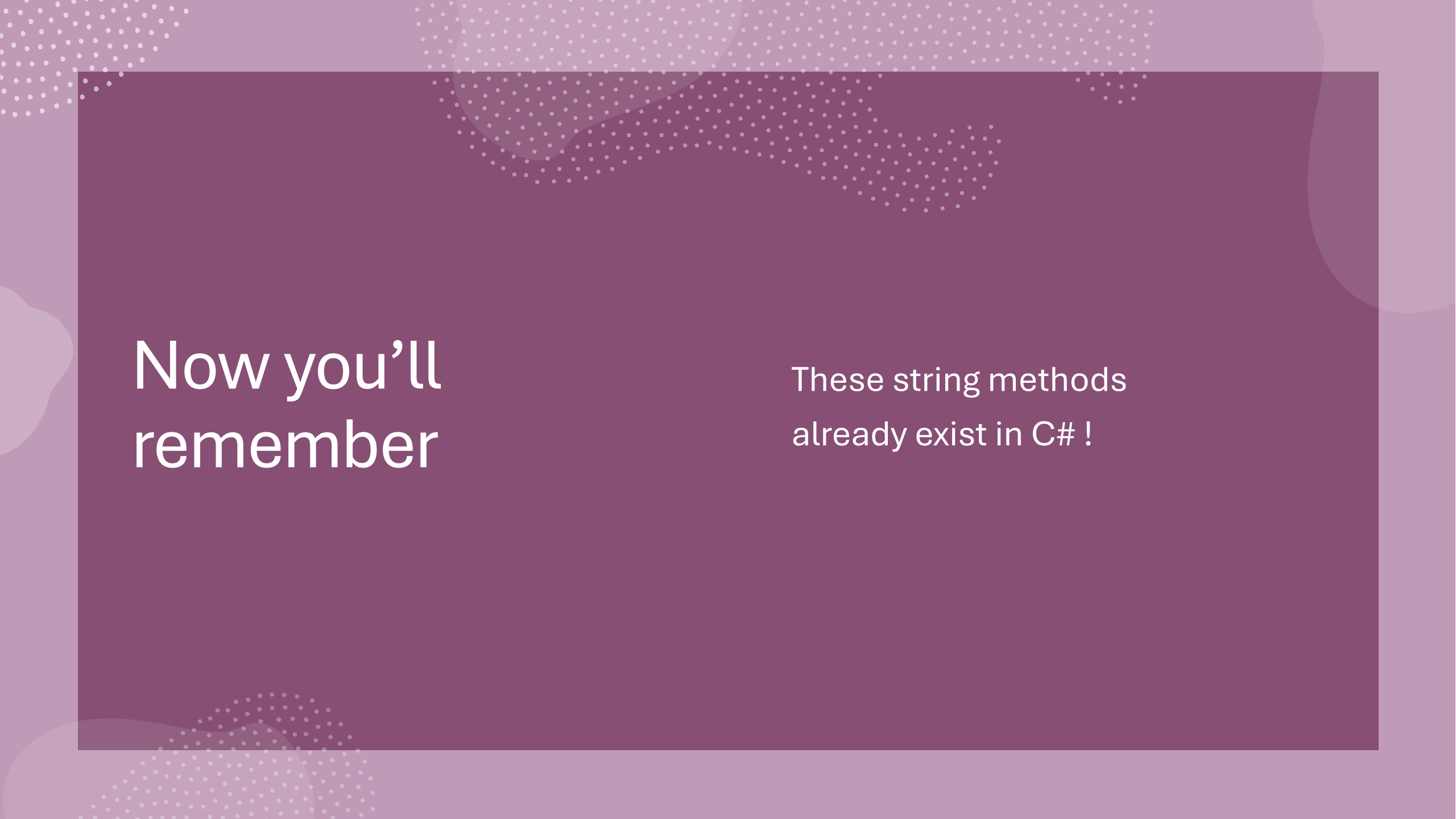
It replaces a substring within a source string with its replacement block.

It assumes the block is contained only once in the string.

If blocco and rimpiazzo have different lengths, it throws an *ArgumentException*.

It returns the replacement string.

Implementation: internally uses «TrovaInizioDi»



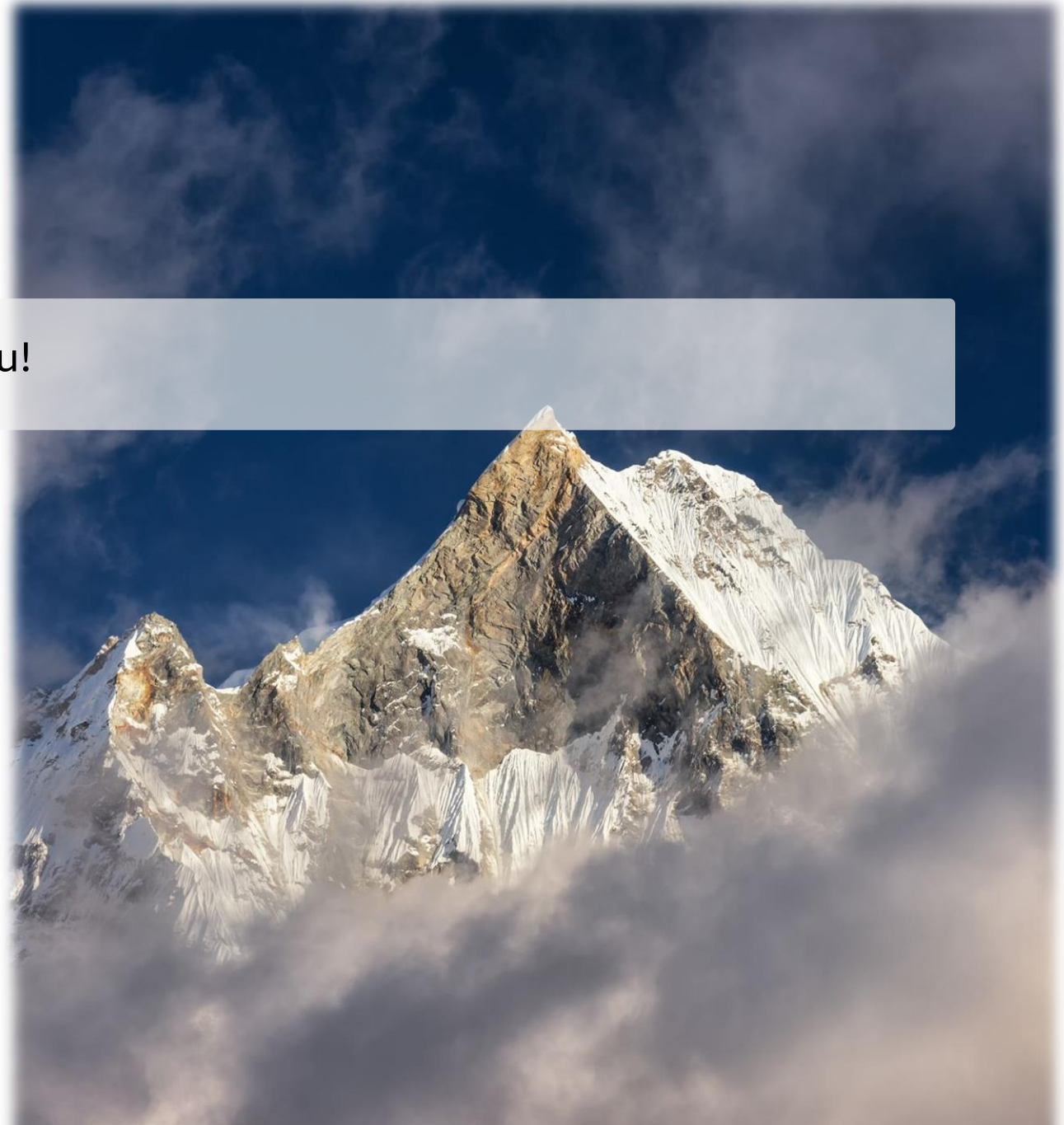
Now you'll remember

These string methods
already exist in C# !

Next Level!

Chapter 4

More complex algorithms, for warriors like you!



Cerca secondo Massimo

Level

MEDIUM

```
public static bool CercaSecondoMassimo(List<int> collezione)
```

Returns the second maximum value in the collection.

Implementation: internally *could* use «CercaMassimo»
It could be generalized to return the number of iterations as parameter.

Cerca Massimi Successivi

Level

MEDIUM

```
public static bool CercaMassimiSuccesivi(List<int> collezione, int iterations)
```

Returns the second maximum value in the collection.

Implementation: internally *could* use «CercaMassimo»
It could be generalized to return the number of iterations as parameter.

Now you'll
remember

CercaMassimi can be
implemented with a
PriorityQueue.

We will see in the next lesson!

Esplora array

Level

MEDIUM

```
public static bool EsploraArray(int[] collezione, int maximumValue)
```

While iterating the array, if the sum of the explored values reaches maximum value, it returns -1 and stop the exploration. Else it return the sum all elements until the end.

Esplora array al contrario

Level

MEDIUM

```
public static bool EsploraArrayAlContrario(int[] collezione, int maximumValue)
```

While iterating the array in backward, if the sum of the explored values reaches maximum value, it returns -1 and stop the exploration. Else it return the sum all elements until the end.

Controlla Pezzi

Level

MEDIUM

```
public static int[] ConfrontaPezzi(int[] supermercato1, int[] supermercato2)
```

For the same element at index i , the two array are containing different pieces of products that are sold in two different supermarkets.

Arrays have same length, and the number of pieces is always available for each product. The method returns a new array containing the minimum piece for each of the available products.

Calcola Sconto

Level

MEDIUM

```
public static decimal[] CalcolaSconto(decimal[] prezzoIngrosso, decimal[] quantita, decimal specialPrice)
```

For the same element at index i , the two array are containing the prices at stock and purchased quantity.

Arrays have same length.

We return an array containing the selling price. The selling price is unit cost plus a margin of 20%.

If the cost is equal to specialPrice, a special discount is applied, and the product is given at price of 0.0.

Confronta Prezzi

Level

MEDIUM

```
public static decimal?[] ConfrontaPrezzi (decimal?[] supermercato1, decimal?[]  
supermercato2)
```

For the same element at index i , the two array are containing different prices at which those products are sold in different supermarkets.

Arrays cannot have same length. For instance, a supermarket can have more products.

Moreover, a correct price is not always available for each product: in that case the array will contain null.

The method returns a new array containing the minimum price for each product.

Valida Punteggio

Level

HARD

```
public static bool ValidaPunteggio (double punteggioArray, double punteggioStringhe)
```

Two *punteggi* must be summed to obtain the total score of the game. The max score of the game is 3. If the total of points equals 0.3 a bonus of 1 is granted.

The method returns true if the total score is equal or above 1. Else false.

Now you'll
remember

*Always be careful when comparing
doubles!*

Better to use $\geq$, $\leq$

Calcola numeri di Fibonacci

Level

HARD

```
public static bool CalcolaNumeroFibonacci(int posizione)
```

Compute the occurrence of Fibonacci number for the given position.

Implementation: it can be built iterating or with recursion. We will build iterating.

Calcola lista di Fibonacci

Level

HARD

```
public static bool CalcolaListaFibonacci(int numeroDiElementi)
```

Return the list of Fibonacci numbers. As many as specified by the parameter.

Implementation: internally uses «CalcolaNumeroFibonacci»

Palindroma

Level

HARD

```
public static bool Palindroma(string sorgente)
```

It returns true if the string is palindrome. Else return false.

Rimpiazza

Level

OMG

```
public static bool Rimpiazza(string sorgente, string blocco, string rimpiazzo)
```

It replaces a substring within a source string with its replacement block.

The block can be contained more than once in the string.

blocco and *rimpiazzo* can have different lengths.

It returns the replacement string.

Implementation: internally uses «TrovaInizioDi»

Let's get back to Turing...

During his life...

- He solved *Enigma* - the german cryptography system – giving a huge contribute to the Allies win in WWII
- He elaborated a test to evaluate the 'intelligence' of a machine (first studies on AI)
- He did studies on mathematical biology and chemistry
- He was also charged of 'homosexuality' and punished with hormonal treatments

How did it end?

He was found on his bed - probably poisoned by lab's smokes.



A **bitten apple** was found in his room, next to his bed.